CLASE
13

Guardar los datos en un archivo

Clase extra: para después del proyecto final

En esta clase vas a aprender

- Por qué los datos se pierden al cerrar el programa.
- Abrir archivos para **leer** y **escribir** con `open()` y `with`.
- Qué es el formato **JSON** y cómo guardar y leer listas con `json.dump()` y `json.load()`.
- Agregarle a tu CRUD un guardado automático, para que los datos **no se pierdan nunca**.

Teoría

El problema: todo se borra al cerrar

Terminaste tu CRUD, cargaste veinte registros… cerrás el programa, lo volvés a abrir y **la lista está vacía**. ¿Por qué? Porque las variables, como la lista `alumnos`, viven en la **memoria RAM** de la computadora, y esa memoria se libera cuando el programa termina.

Para que los datos duren, hay que **guardarlos en un archivo** en el disco, igual que cuando guardás un documento de Word. Al abrir el programa, los **leemos** de ese archivo y los volvemos a poner en la lista.

	Memoria (variables)	Disco (archivos)
¿Qué tan rápido es?	Muy rápido	Más lento
¿Qué pasa al cerrar el programa?	Se borra todo	Los datos quedan guardados
¿Cómo se usa en Python?	`alumnos = [...]`	`open(...)`, `json.dump(...)`

Abrir un archivo: open() y with

La función `open()` abre un archivo. Recibe el **nombre** del archivo y el **modo**, que indica para qué lo abrimos:

Modo	Significa	Qué hace
`"r"`	read (leer)	Lee el archivo. Si no existe, da el error `FileNotFoundError`.
`"w"`	write (escribir)	Escribe el archivo. Si no existe, lo crea. **Si existe, lo borra y lo escribe de nuevo.**
`"a"`	append (agregar)	Agrega al final del archivo, sin borrar lo que había.

Siempre lo vamos a usar junto con `with`. La ventaja de `with` es que **cierra el archivo solo** cuando termina el bloque con sangría, aunque haya un error. Además agregamos `encoding="utf-8"` para que los acentos y la ñ se guarden bien.

```
# Escribir un archivo de texto
with open("saludo.txt", "w", encoding="utf-8") as archivo:
    archivo.write("¡Hola desde Python!\n")
    archivo.write("Esta es la segunda línea.\n")

# Leerlo
with open("saludo.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()

print(contenido)
```

Ejecutalo y después buscá el archivo `saludo.txt` en la carpeta: lo podés abrir con el Bloc de notas. Dentro del bloque `with`, la variable `archivo` representa al archivo abierto: `write()` escribe texto y `read()` lo lee todo.

El formato JSON

Con `write()` solo se puede guardar **texto**. Pero nuestros datos son una **lista de listas**, con textos, números y valores `True` / `False`. Para guardarlos sin tener que convertir todo a mano usamos **JSON**, un formato de texto que se parece muchísimo a Python. Es uno de los formatos más usados del mundo para guardar y compartir datos: lo usan las aplicaciones del celular, las páginas web y los videojuegos.

Así se ve la lista de alumnos de la clase 12 guardada en JSON:

alumnos.json

```
[
  ["Ana", "García", 8.0],
  ["Luis", "Martínez", 6.5],
  ["Sofía", "López", 9.0]
]
```

¡Es casi igual a escribirla en Python! Las diferencias son pocas:

En Python	En JSON	Ejemplo en JSON
Lista `[ ]`	Arreglo `[ ]` (igual)	`[1, 2, 3]`
Texto `"..."` o `'...'`	Texto, **solo con comillas dobles**	`"Ana"`
Números `8` y `6.5`	Números (igual)	`8`, `6.5`
`True` / `False`	`true` / `false` (en minúscula)	`true`
`None`	`null`	`null`

json.dump() y json.load()

El módulo `json` viene incluido con Python. Tiene dos funciones que hacen todo el trabajo:

- `json.dump(datos, archivo)`: **guarda** los datos de Python en un archivo abierto con `"w"`.
- `json.load(archivo)`: **lee** un archivo abierto con `"r"` y **devuelve** los datos como lista de Python.

```
import json

alumnos = [
    ["Ana", "García", 8.0],
    ["Luis", "Martínez", 6.5],
    ["Sofía", "López", 9.0],
]

# Guardar
with open("alumnos.json", "w", encoding="utf-8") as archivo:
    json.dump(alumnos, archivo, indent=2, ensure_ascii=False)

# Leer (como si fuera otro programa, o el mismo al día siguiente)
with open("alumnos.json", "r", encoding="utf-8") as archivo:
    leidos = json.load(archivo)

print(leidos)           # [['Ana', 'García', 8.0], ['Luis', 'Martínez', 6.5], ...]
print(leidos[1][0])     # Luis  -> se usa igual que cualquier lista
```

Las dos opciones extra de `dump` no son obligatorias, pero conviene usarlas siempre:

- `indent=2` acomoda el archivo en varios renglones con sangría, para que una persona lo pueda leer.
- `ensure_ascii=False` guarda los acentos y la ñ tal cual. Sin esta opción, "García" quedaría escrito como `"Garc\u00eda"`.

¿Y si el archivo no existe?

La **primera vez** que se abre el programa, el archivo todavía no existe, y `open(..., "r")` da el error `FileNotFoundError`. Además, si alguien edita el archivo a mano y lo rompe (por ejemplo, borra un corchete), `json.load` da el error `json.JSONDecodeError`. Ya sabemos cómo atrapar errores con `try` / `except` (clase 5), y un mismo `try` puede tener **varios `except`**, uno para cada tipo de error:

```
def cargar():
    try:
        with open(ARCHIVO, "r", encoding="utf-8") as archivo:
            return json.load(archivo)
    except FileNotFoundError:
        return []                       # primera vez: empezamos con la lista vacía
    except json.JSONDecodeError:
        messagebox.showerror("Error", "El archivo de datos está dañado")
        return []
```

Al guardar también puede fallar algo (por ejemplo, si la carpeta no permite escribir). Ese error se llama `OSError`:

```
def guardar():
    try:
        with open(ARCHIVO, "w", encoding="utf-8") as archivo:
            json.dump(tareas, archivo, indent=2, ensure_ascii=False)
    except OSError:
        messagebox.showerror("Error", "No se pudo guardar el archivo")
```

¿Dónde se guarda el archivo?

Si escribimos solo `open("tareas.json", ...)`, Python lo busca en la **carpeta de trabajo**, que no siempre es la carpeta del programa: depende de si lo ejecutás con IDLE, con VS Code o con doble clic. Para que el archivo quede **siempre al lado de tu programa**, armamos la ruta completa con el módulo `os`:

```
import os

ARCHIVO = os.path.join(os.path.dirname(os.path.abspath(__file__)), "tareas.json")
```

No hace falta memorizar esta línea: copiala tal cual y cambiá solo el nombre del archivo. Lo que hace es tomar la ubicación de tu programa (`__file__`), quedarse con su carpeta y agregarle el nombre del archivo.

Ejemplo completo: lista de tareas que no se olvida

Retomamos la lista de tareas de la clase 9 y le agregamos el guardado. La idea es la misma que usamos en el CRUD:

- **Al abrir el programa**: `tareas = cargar()` y después `refrescar()`.
- **Después de cada cambio**: guardar. Como ya llamamos a `refrescar()` después de cada cambio, alcanza con poner `guardar()` **adentro de `refrescar()`**.

```
import tkinter as tk
from tkinter import messagebox
import json
import os

# =============== DATOS ===============
ARCHIVO = os.path.join(os.path.dirname(os.path.abspath(__file__)), "tareas.json")
tareas = []

# =============== ARCHIVO ===============
def guardar():
    try:
        with open(ARCHIVO, "w", encoding="utf-8") as archivo:
            json.dump(tareas, archivo, indent=2, ensure_ascii=False)
    except OSError:
        messagebox.showerror("Error", "No se pudo guardar el archivo")

def cargar():
    try:
        with open(ARCHIVO, "r", encoding="utf-8") as archivo:
            return json.load(archivo)
    except FileNotFoundError:
        return []
    except json.JSONDecodeError:
        messagebox.showerror("Error", "El archivo de tareas está dañado")
        return []

# =============== FUNCIONES ===============
def refrescar():
    lst_tareas.delete(0, tk.END)
    for tarea in tareas:
        lst_tareas.insert(tk.END, tarea)
    lbl_total.config(text=f"Tareas guardadas: {len(tareas)}")
    guardar()                       # después de cada cambio, se guarda solo

def agregar():
    texto = txt_tarea.get().strip()
    if texto == "":
        messagebox.showwarning("Atención", "Escribí una tarea")
        return
    tareas.append(texto)
    refrescar()
    txt_tarea.delete(0, tk.END)
    txt_tarea.focus()

def eliminar():
```

```
    seleccion = lst_tareas.curselection()
    if len(seleccion) == 0:
        messagebox.showwarning("Atención", "Seleccioná una tarea de la lista")
        return
    tareas.pop(seleccion[0])
    refrescar()

# =============== VENTANA ===============
ventana = tk.Tk()
ventana.title("Mis tareas")

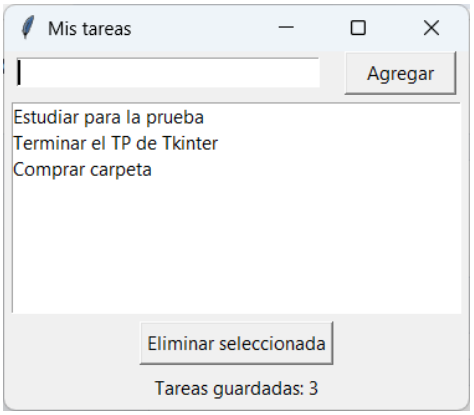
txt_tarea = tk.Entry(ventana, width=30)
txt_tarea.grid(row=0, column=0, padx=5, pady=5)
tk.Button(ventana, text="Agregar", width=10, command=agregar).grid(row=0, column=1,
padx=5)

lst_tareas = tk.Listbox(ventana, width=45, height=8)
lst_tareas.grid(row=1, column=0, columnspan=2, padx=5, pady=5)

tk.Button(ventana, text="Eliminar seleccionada", command=eliminar).grid(row=2, column=0,
                                                                        columnspan=2)
lbl_total = tk.Label(ventana, text="")
lbl_total.grid(row=3, column=0, columnspan=2, pady=5)

tareas = cargar()          # al abrir: leemos lo que había guardado
refrescar()
txt_tarea.focus()
ventana.mainloop()
```

Así se ve al ejecutarlo:



Probalo así: agregá tres tareas, **cerrá el programa** y volvé a abrirlo. ¡Las tareas siguen ahí! Abrí también el archivo `tareas.json` con el Bloc de notas para ver cómo quedó.

Fijate en que `tareas = cargar()` va **después** de crear la ventana: si el archivo estuviera dañado, `messagebox` necesita que la ventana ya exista para mostrar el error.

Y en el CRUD de la clase 12

Para agregarle el guardado al CRUD de alumnos hacen falta solo **cinco cambios**:

Dónde	Qué agregar
Arriba de todo	`import json` y `import os`
En la parte de DATOS	La línea de `ARCHIVO = ...` con el nombre `"alumnos.json"`
En la parte de FUNCIONES	Las funciones `guardar()` y `cargar()`, cambiando `tareas` por `alumnos`
Al final de `refrescar()`	Una llamada a `guardar()`
Antes de `mainloop()`	`alumnos = cargar()` seguido de `refrescar()`

Para recordar

- Las variables se borran al cerrar el programa; los **archivos** quedan guardados en el disco.
- Usá siempre `with open(ARCHIVO, modo, encoding="utf-8") as archivo:`.
- Modo `"w"` para guardar (reemplaza todo el archivo) y `"r"` para leer.
- `json.dump(datos, archivo)` guarda, `json.load(archivo)` lee y devuelve la lista.
- Atrapá `FileNotFoundError` (no existe todavía) y `json.JSONDecodeError` (archivo dañado).

⚠ ¡Atención!

el modo `"w"` **borra todo lo que había** en el archivo antes de escribir. Por eso siempre hay que guardar la lista **completa**, no solo el último registro. Y si el archivo estaba dañado, la próxima vez que se guarde se va a reemplazar por la lista nueva: si querés conservar el archivo roto para revisarlo, hacé una copia antes.

Práctica

Ejercicio 13.1 ★☆☆ **En consola**: creá una lista con 5 películas y guardala en `peliculas.json` con `json.dump`. Abrí el archivo con el Bloc de notas y fijate cómo quedó. Probá guardarlo con y sin `indent=2`: ¿qué diferencia hay?

Ejercicio 13.2 ★☆☆ **En consola**: hacé otro programa que lea `peliculas.json` con `json.load` y muestre las películas numeradas con un `for`.

Ejercicio 13.3 ★★☆ Escribí el ejemplo de la lista de tareas y probá estos casos: (a) agregar tareas, cerrar y volver a abrir; (b) borrar el archivo `tareas.json` y abrir el programa; (c) abrir el archivo con el Bloc de notas, borrar el último corchete `]`, guardar y abrir el programa. Anotá qué pasa en cada caso: _____

Ejercicio 13.4 ★★☆ Agregale el guardado al **CRUD de alumnos** de la clase 12 siguiendo los cinco cambios de la tabla. Comprobá que, después de cerrar y abrir, las notas siguen siendo números (por ejemplo, que "Por nota" ordene bien).

Ejercicio 13.5 ★★★ Cambiá el CRUD para que **no** guarde solo: agregá un botón **"Guardar"** y una variable `hay_cambios` que se ponga en `True` cada vez que se agrega, modifica o elimina algo. Al tocar "Salir", si hay cambios sin guardar, preguntá con `askyesno` si se quieren guardar antes de cerrar.

🛠 Mi proyecto: paso a paso

- Agregale a tu proyecto el guardado automático en un archivo JSON con el nombre de tu tema (por ejemplo `libros.json`).
- Si tu proyecto empezaba con registros de ejemplo, hacé que solo se usen cuando el archivo todavía no existe (en el `except FileNotFoundError:`).
- Probá los tres casos del ejercicio 13.3 con tu proyecto: cerrar y abrir, archivo borrado y archivo dañado.